

A Multi-Engine Dataflow for MoE Decoding on Scratchpad-Based Tensor Accelerators

Bin Ma
University of California, Merced
Merced, California, USA
bma100@ucmerced.edu

Wenjie Fan
Yotta Labs
USA
fanwj@mail.usc.edu.cn

Dong Li
University of California, Merced
Yotta Labs
Merced, California, USA
dli35@ucmerced.edu

Abstract

Mixture-of-Experts (MoE) decoding on scratchpad-based tensor accelerators (STA) is dominated by moving expert weights while the compute engines sit idle. This traffic is hard to hide, because the experts are known only after routing, and hard to shrink without losing quality or adding critical-path work. We present CARDAN, which represents each expert-weight matrix as a vector-quantized component plus a shared-basis low-rank component and co-designs this representation with a multi-engine decoding dataflow. The representation separates expert-common from expert-private work, so the dataflow overlaps DMA with computation on several engines. Across five MoE families on AWS Trainium3, CARDAN matches or improves BF16-teacher perplexity across all five models and speeds up batch-one decoding by 1.15–1.31 $\times$ over AWS dense MoE megakernels, rising to 1.7 $\times$ at batch size 16.

1 Introduction

Mixture-of-Experts (MoE) scales language models by activating only a subset of a layer’s experts for each token, increasing parameter capacity while keeping per-token floating-point operations (FLOPs) nearly constant. MoE has therefore become a common architecture in recent frontier models.

The scratchpad-based tensor accelerator (STA) is a specialized hardware system designed for deep learning and matrix workloads. STA relies on a software-controlled Scratchpad Memory (SPM) rather than a traditional hardware managed cache: software places each tensor explicitly on SPM, and Direct Memory Access (DMA) engines move data between high-bandwidth memory (HBM) and SPM. By using explicitly managed on-chip storage, STA eliminates cache tag overhead and unpredictable replacement policies, maximizing energy efficiency and throughput for large-scale mathematical computations. STA has been exemplified by a range of accelerators, such as AWS Trainium [3], Google TPU [20], Intel Gaudi 2 [15], and Huawei Ascend [18]. Hence, we study MoE on STA.

Problems. Despite a sparse routing mechanism that activates only a subset of experts per token, MoE decoding latency on STAs is still often dominated by expert-weight DMA. For example, Qwen3-30B-A3B has 48 layers, and each layer routes a token to activate only eight of 128 experts,

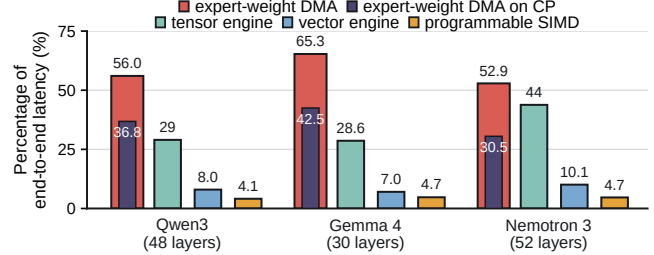


Figure 1. The breakdown of decoding latency for three MoE models from Hugging Face [14, 29, 36]. The decoding generates one token based on four-way tensor parallelism. “CP” stands for critical path.

generating 75 MB of weight traffic (using BF16) per layer per token. The acting expert set may change from token to token, forcing the selected expert weights to be loaded from HBM with little reuse. We run this Qwen3 decoding with four-way tensor parallelism on an example STA, AWS Trainium3 using four logical NeuronCores, each with 64 MiB of SPM. Figure 1 shows the results. For Qwen3, Loading expert weights takes 56% of the end-to-end latency, and the loading time is largely (36.8%) exposed to the critical path. We have similar observations on Gemma 4 and Nemotron 3: the loading time exposed to the critical path takes a large portion (42.5% and 30.5%) of the end-to-end latency.

Addressing the long latency problem due to expert-weight movement faces two challenges. First, expert-weight traffic is difficult to hide. Prefetch, by overlapping with compute, is an effective approach to hide the data movement overhead, but the experts to prefetch is only known after the router selects experts, which brings limited opportunities for overlap. The recent efforts [9, 19, 40, 47] anticipate the routing selection by utilizing history of expert activations. However, those efforts cannot work well on STA, because staging expert weights before routing creates a long-lived buffer in limited SPM, and disrupts the effectiveness of compiler optimization to maximize data locality. Even worse, the compiler optimization can reclaim and overwrite the memory of prefetched experts in SPM [2], invalidating prefetch and wasting data movement.

Second, reducing the traffic size to reduce the data movement overhead is challenging. Changing the expert-weight representation (e.g., using sparse tensor formats [7, 34]) can reduce traffic size, but easily disrupts the effectiveness of DMA. The DMA service time depends not only on traffic size,

but also on the number of DMA packets, memory address regularity, and transfer serialization [4, 39]. A smaller weight-representation may replace regular transfers with more finer-grained or irregular ones, and transferring fewer bytes does not proportionally translate to shorter DMA-service time. Existing efforts [7, 34] also show that compressed tensor formats can introduce metadata-dependent memory accesses and that the format with the smallest storage footprint can incur longer execution latency. Moreover, some tensor representations trade traffic reduction with additional runtime work. For example, vector quantization (VQ) [21, 27] reduces weight traffic but requires that each expert stores an index map into a code book. Selecting the code word in the code book introduces runtime overhead, taking up to 20% of the end-to-end decoding latency in our evaluation on Trainium.

Besides the data movement problem, we also see under-utilized compute engines when serving MoE decoding on STA. The STA is often characterized with multiple compute engines, such as tensor engine and SIMD engine, each of which is good at handling workloads with specific properties (e.g., memory access patterns and tensor shapes). Take Qwen3 shown in Figure 1 as an example again. The SIMD engine is active only during 4.1%-4.7% of serving time, and the vector engine is active only during 7%-10.1%.

However, decomposing the MoE workload and map it to those engines is challenging, because routing, expert-weight transfer, and expert computation form a strict dependency chain, leaving little concurrency to tap without losing MoE quality. Furthermore, decomposing the expert computation (including expensive up- and down-projection) to finer-grained computation tasks and mapping them across engines cannot bring latency reduction because of operand dependencies across engines and extra SPM traffic.

Key insights. We reveal that designing an STA-customized representation for expert weights is the key to address the above problems of data movement and under-utilized hardware. Co-designing the weight representation with the execution dataflow reduces HBM-to-SPM weight transfers while exploiting multiple heterogeneous compute engines to work concurrently with DMA transfers.

Solutions. Based on the above insights, we introduce CARDAN, an MoE framework that employs a STA-customized expert weight representation and leverages multiple compute engines in STA. CARDAN has multiple innovations.

First, CARDAN uses a “hybrid” expert-weight representation. To reduce the data movement, we employ the low rank (LR) factorization method [5, 28]. This method was originally used to reduce the memory consumption of MoE parameters by extracting a common basis-tensor per layer from all experts in the same layer; besides the basis tensor, each activated expert uses small, private coefficients to reconstruct the expert. To reduce the data movement, at each layer, we put the basis tensor in SPM and *share* it across experts, but sparsely load the small expert-private coefficients.

However, we find that the LR leads to severe quality loss, because the expert-private coefficients only provide low-rank approximation of expert weights and miss critical expert-specific information. Using the LR, for example, Qwen3-30B-A3B loses the quality: the perplexity, which is the output quality metric and whose lower is better, increases from 9.82 to 34.91. Hence, we face a tradeoff between reducing data movement and maintaining MoE quality.

To avoid the quality loss, we use a VQ technique [26] in combination with the LR. The VQ fits STA naturally, because the VQ is based up on a common and small code book shared across experts in a layer. As a result, the code book can reside in SPM, loaded once but reused often, which avoids data movement. Using the VQ, the experts are re-constructed by selectively loading code word from the code book, restoring some expert information missed in the low rank. Meanwhile, the VQ benefits from the LR for reducing the runtime overhead, because the combination of the VQ and LR provides sufficient quality, which allows the VQ to use coarse-grained code words to reduce runtime overhead. As a result, the hybrid LR/VQ brings three-fold benefits: less data movement, low runtime overhead, and high quality. Also, since the basis tensor and code book are small, the compiler has much space in SPM to tap and improve tensor locality, and there is no conflict for performance optimization.

Second, CARDAN leverages the new expert-weight representation to enable higher utilization of multiple compute engines. Based upon the hybrid expert-weight representation, we are able to break the traditional routing-transfer-compute dependency chain to two parallel phases: one mainly focusing on expert-common computation without the dependency on the routing decision and the other focusing on expert-private computation. Each parallel phase has multiple independent compute and IO tasks, bringing parallelism and overlap of compute/IO. Also, the tasks in each phase have different characteristics and hence enable efficient utilization of different engines.

We deploy CARDAN across Qwen3, DeepSeek-V2-Lite, Nemotron, OLMoE and Gemma4 on AWS Trainium3 (an example of STA). Across these deployments, CARDAN accelerates decoding by 1.15–1.31 $\times$ over the state-of-the-art baseline of AWS BF16 dense megakernel, while achieving similar quality. Our work spans computer architecture and operating system (generally defined), with contributions to heterogeneous architectures and dataflow runtime system. We summarize the major contributions as follows.

- We analyze the MoE performance on STA, and identify the data movement as the most significant challenge of deploying MoE on promising STA architectures.
- We create a new method to reduce and hide the data movement overhead on STA based on a hybrid expert-weight representation. This method breaks the traditional

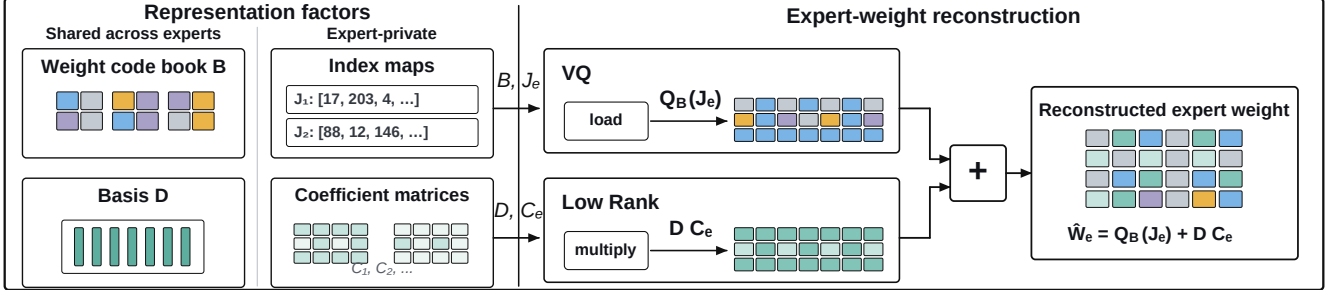


Figure 4. Hybrid expert-weight representation. Shared $\mathcal{B}$ and D combine with expert-private J_e and C_e to form VQ and LR whose sum represents $\widehat{W}_e$.

2.3 Expert Weight Representations

The existing efforts use the LR and VQ to save memory. We review them as follows.

Low rank (LR) method factorizes a given matrix to two lower-rank matrices. In the context of MoE, the LR can approximate the expert-weight matrix of all experts in a layer (i.e., W) as a product of two low-rank matrices: $W \approx DC$, where $D \in \mathbb{R}^{H \times r}$ is the basis tensor (rank- r) and shared across the experts in the same layer and C is a collection of expert-private coefficients [5, 28]. As a result, for each expert e , its weight matrix $W_e \approx DC_e$ where $C_e \in \mathbb{R}^{r \times I}$ is an expert-private coefficient matrix. C_e , while preserving expert-private information, cannot recover information discarded by the shared projection $D^\top$.

Vector quantization (VQ) is built upon a set of representative vectors, each of which is called a code word. The vector set is called the code book containing K code words. Given a vector, VQ approximates it by one of the code words in the code book. Hence, instead of storing the vector itself, VQ stores only an integer index that points to a specific code word in the code book. As a result, the vector is recovered by a single table lookup [26].

We apply a blockwise form of VQ to the expert weight matrix of all experts [21]: each column of the expert-weight matrix is partitioned into blocks, each of which is length- d , and each weight block is replaced by a code word. Given a weight block, the position (or index) of its code word in the code book (called the *weight code book* in the remaining of the paper) is stored in an index map. Experts in the same layer share the weight code book but keep separate index maps, allowing each expert to use a different combination of code words. A larger d reduces the numbers of weight blocks and indices, but generally increases approximation error because each code word represents a larger block.

3 Hybrid Expert-Weight Representation

CARDAN combines LR and VQ to reduce critical-path expert-weight traffic without degrading model quality or creating large runtime overhead.

3.1 Hybrid Expert-Weight Representation

CARDAN represents each weight matrix of gate and up projection as the sum of LR and VQ. In the existing efforts, both LR and VQ were individually proposed to shrink model memory (Section 2.3); CARDAN combines them for a different purpose – reducing expert-weight movement on the critical path of decoding on STA and increasing utilization of the heterogeneous compute engines. In CARDAN, given $W_e \in \mathbb{R}^{H \times I}$ and input activation h , the expert computation performs the following.

$$o_e = W_e^\top h \approx \widehat{W}_e^\top h = \underbrace{Q_{\mathcal{B}}(J_e)^\top h}_{\text{VQ}} + \underbrace{C_e^\top (D^\top h)}_{\text{LR}}. \quad (1)$$

where J_e is expert e 's private index map into the weight code book, and $Q_{\mathcal{B}}(J_e)$ is the resulting $H \times I$ VQ matrix obtained from these weight code book lookups. The weight code book $\mathcal{B}$ and LR basis tensor D are shared across experts within each layer and projection: within a layer, the gate and up-projection weights have separate weight code books $\mathcal{B}$, and have the LR's basis tensor D shared across experts. Meanwhile, each expert holds private J_e and C_e for each of the gate and up projections. Section 3.2 discusses why the LR and VQ complement each other. During decoding, the layer-shared $\mathcal{B}$ and D are loaded once per token, and each activated expert adds only its J_e and C_e to the expert-weight payload.

We do not apply LR and VQ to the down projection and keep the down-projection weight matrix in its original BF16, because of two reasons. First, all activated experts apply their gate and up projection weights to the same h , allowing the work involving the shared $\mathcal{B}$ and D to be reused; the down projection weights receive expert-specific intermediate activation p_e from gate and up projection. Applying CARDAN's VQ + LR representation to the down-projection weights would therefore multiply $\mathcal{B}$ and D by a different p_e for every activated expert. Second, over 80% of the DMA time for down-projection weights overlaps gate/up computation, leaving little on the critical path.

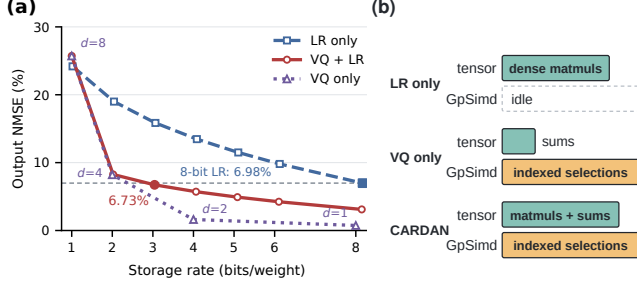


Figure 5. The accuracy of hybrid weight representation for Qwen3-30B-A3B, and work placement. (a) Gate/up-projection output NMSE versus storage rate, averaged over Layers 0, 24, and 47. (b) engines that executes each representation.

3.2 Representation Complementarity

The LR and VQ complement each other in CARDAN. We use Qwen3-30B-A3B (48 layers in total) as an example to depict why CARDAN needs both. We quantify output quality using the metric, normalized mean squared error (NMSE) at gate and up projections of experts at Layers 0, 24, and 47. In our study, NMSE measures the difference between a weight representation (LR only, VQ only, or LR+VQ) and the BF16 output (the original output), defined as follows.

$$\text{NMSE} = \frac{\sum \|\widehat{W}_e^\top h - W_e^\top h\|_2^2}{\sum \|W_e^\top h\|_2^2}, \quad (2)$$

where $\widehat{W}_e$ is the weight reconstructed by LR only, VQ only, or VQ+LR, and the summation terms run over evaluation tokens h , experts, and both gate and up projection within a layer, and report the mean of the three per-layer NMSE values for Layers 0, 24, and 47. A lower NMSE indicates that the output is closer to the original BF16 output, and vice versa. Figure 5(a) plots NMSE against storage rate (i.e., the number of bits used per weight).

Figure 5(a) compares the three representations. At approximately 4 bits/weight, NMSE is 13.46% for LR only, 5.72% for VQ+LR, and 1.62% for VQ only ($d = 2$). LR only at 8 bits/weight (6.98%) is still worse than VQ+LR at 3 bits/weight (6.73%), so VQ+LR needs less than half the storage for the same accuracy. VQ only is more accurate, but only by shrinking d , and every halving of d doubles the indexed selections per weight, making indexed selection a major overhead on critical path.

Workload placement on STA. The LR and VQ are also complementary in terms of the placement on heterogeneous compute engines, figure 5(b) shows this complementarity. The LR computes $z = D^\top h$ once per input and $C_e^\top z$ for each activated expert using dense matrix-vector multiplications on the tensor engine. The VQ uses each expert’s indices J_e to select code words from the projection code book in SBUF (Section 4.2): one indexed selection per weight block. The indexed selections for weights are independent from each other and use less compute, hence performed on GpSimd.

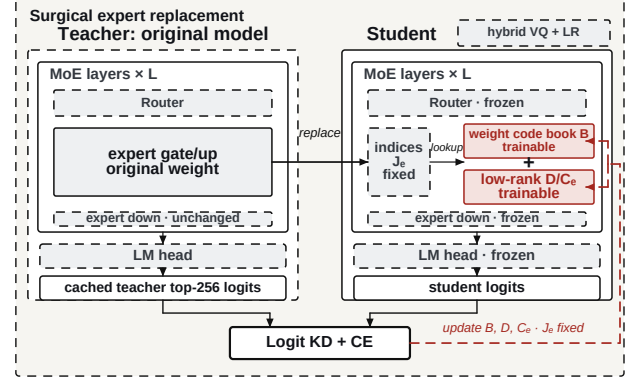


Figure 6. Training scope. Only the gate/up weight code book, LR basis tensor D and coefficient matrix C_e are trained.

In addition, the LR computation and VQ lookup happen in parallel because neither requires the other’s output.

After the gate/up-projection outputs are formed, the scalar engine applies model-specific expert activation ϕ to the gate-output vector. The vector engine then multiplies corresponding elements of the activated gate-output vector and the up-projection-output vector, producing the intermediate activation p_e for the down projection.

3.3 Initialization of Hybrid Representation

Building the hybrid representation must initialize the weight code book $\mathcal{B}$, the index maps J_e , the LR basis tensor D , and the coefficient matrices C_e , so that the output of expert computation and the final output of the model match those of the original output in BF16. CARDAN first applies a training-free, layer-wise initialization to obtain a working representation without any gradient training. A short end-to-end distillation then restores the model quality to the BF16 level. (Figure 6).

Training-free initialization. We initialize the VQ using the standard product quantization [21], applied separately to the gate and up weights of each layer. This initialization produces the shared weight code book per layer $\mathcal{B}$ and the expert-private index maps J_e [21]. Then, we fit the LR in closed form on representative decode activations, so the fit prioritizes the errors that affect expert outputs [45]. The fit obtains one D shared across experts in a layer and a separate C_e for each expert, following standard LR correction methods [38, 48, 51]. Using the closed-form fit is beneficial, because it provides a usable initialization without gradient-based training or learning-rate tuning. Appendix A formalizes the objective for a layer and derives its solution.

End-to-end distillation. The layer-wise initialization minimizes each layer’s own output error, but it cannot see how errors are accumulated and propagated across the layers. To solve this problem, we use knowledge distillation (KD) that jointly refines all layers [10, 27, 42]. In particular, the teacher is the original BF16 model with all weights frozen,

and the student is the same model with the hybrid representation in place of the expert gate and up weights. For each token, Ω denotes the set of vocabulary indices of the teacher’s 256 largest logits. We take the teacher and student logits at the indices in Ω , divide both by a scaling factor τ , and apply softmax, yielding p_{tea}^Ω and p_{stu}^Ω . With $\mathcal{L}_{\text{CE}}$ denoting next-token cross-entropy, we minimize the following.

$$\mathcal{L} = \tau^2 D_{\text{KL}}(p_{\text{tea}}^\Omega \| p_{\text{stu}}^\Omega) + \lambda \mathcal{L}_{\text{CE}}. \quad (3)$$

where λ is a model-quality alignment hyperparameter. During KD, only $\mathcal{B}$, D , and C_e in the student are updated; J_e and all other model parameters are frozen (Figure 6).

4 Multi-Engine Decoding Dataflow

The weight representation in Section 3 reduces memory traffic and introduces work across multiple compute engines. CARDAN organizes the decoding work into a multi-engine dataflow which breaks the routing–transfer–compute dependency chain and overlaps data movement with computation. Figure 7 summarizes the resulting dataflow.

4.1 Multi-Engine Overlap Schedule

Section 3.2 places the LR tensor multiplications on the tensor engine and the VQ selections on GpSimd. This subsection describes *when* that work and the associated DMA transfers are issued, so that the DMA transfer is hidden along the routing–transfer–compute chain (Figure 7, top). CARDAN separates routing-independent shared work (i.e., loading $\mathcal{B}$ and D and multiplying them by h) from routing-dependent expert-private work (i.e., loading and applying J_e , C_e , and the down projection). This separation exposes three windows to overlap the data transfer and compute.

- The shared code book $\mathcal{B}$ and LR basis tensor D do not depend on which experts are activated, so the DMA transfers of $\mathcal{B}$ and D can be issued while the router is computing the expert scores and selecting top- k experts.
- After the routing, the tensor engine performs two operations: (1) the rank- r projection $z = D^\top h$ and (2) multiplying each length- d block of h with K code words of $\mathcal{B}$. We call the two operations *the shared projections*. Meanwhile, DMA loads the activated experts’ J_e and C_e .
- GpSimd performs the J_e -indexed selections while the tensor engine computes $C_e^\top z$. The addresses of the selected down-projection weights are known after the routing, and do not depend on the gate/up-projection outputs, so the DMA transfer of those weights can be issued during the computation on the GpSimd and tensor engine.

4.2 Removing Redundancy in Compute and Memory

The multi-engine dataflow has redundancy in compute and memory: (1) A straightforward execution of the hybrid representation would reload B and D and recompute the shared

projections for each expert. (2) In VQ, GpSimd uses each expert’s index map to access the code book. GpSimd repeatedly pays the setup cost of issuing selection instructions, leading to redundancy in GpSimd compute. (3) The parallelization of LR and VQ leads to independent stores of the LR/VQ results in SBUF, leading to memory redundancy during the summation of the LR and VQ results. We use the following techniques to remove the redundancy.

Reuse of shared projections. CARDAN performs the shared projections only once *before* the expert computation, rather than once per expert during the expert computation. As a result, each expert can reuse the shared projections, removing the repeated loading of D and B , and matmul for the projection. In particular, before the expert computation, the tensor engine performs $z = D^\top h$ for LR, to be shared between experts; also, the tensor engine multiplies every length- d block h_b of h with K code words in the weight code book $\mathcal{B}$, producing a *projection code book* $T[b, k] = h_b^\top \mathcal{B}_k$. During the expert computation, instead of using the weight code book, an expert selects code words from the projection code book T to select code words.

Both z and T stay in SBUF from the stage 1 until all activated experts have used them (see Figure 7). The book T is small. It holds KH/d values (in BF16) per gate- or up-projection. For example, with $K=256$ and $H=2048$, the two projection-code-books for the gate- and up-projection together occupy only 0.5 MiB at $d=4$ or 0.25 MiB at $d=8$.

Cross-expert gather batching. The VQ uses each expert’s index map J_e to pick code words from T . A straightforward implementation would issue one indexed gather per activated expert. Since each gather pays a constant setup cost on GpSimd, the setup costs of all activated experts add up on the critical path.

CARDAN concatenates J_e of the activated experts and passes the concatenation as a batch to the NKI ISA primitive `nc_n_gather`. Within each of the gate and up projections, since the activated experts select from the same T concurrently using their own index maps, batching preserves every lookup while paying the constant gather setup once per batch, rather than once per expert. Furthermore, because J_e is loaded separately for every activated expert, CARDAN stores the indices in J_e as `uint8`, which limits expert-private HBM traffic and addresses exactly the $K=256$ code words. This solution is depicted in the stage 2 in Figure 7.

Joint accumulation of LR and VQ. The LR and VQ reach the tensor engines in different forms. The VQ provides gathered code words that must be reduced, whereas the LR computes the dense product $C_e^\top z$. Their sum is the gate or up projection output o_e in Eq. (1), and the expert activation consumes only this summation. Materializing the LR and VQ independently would create separate spaces in SBUF, and require an additional read-add-write before activation. As shown in the stage 3 in Figure 7, CARDAN maps the addition of the LR and VQ onto Trainium3’s PSUM for accumulation.

CARDAN decode dataflow in one logical NeuronCore program

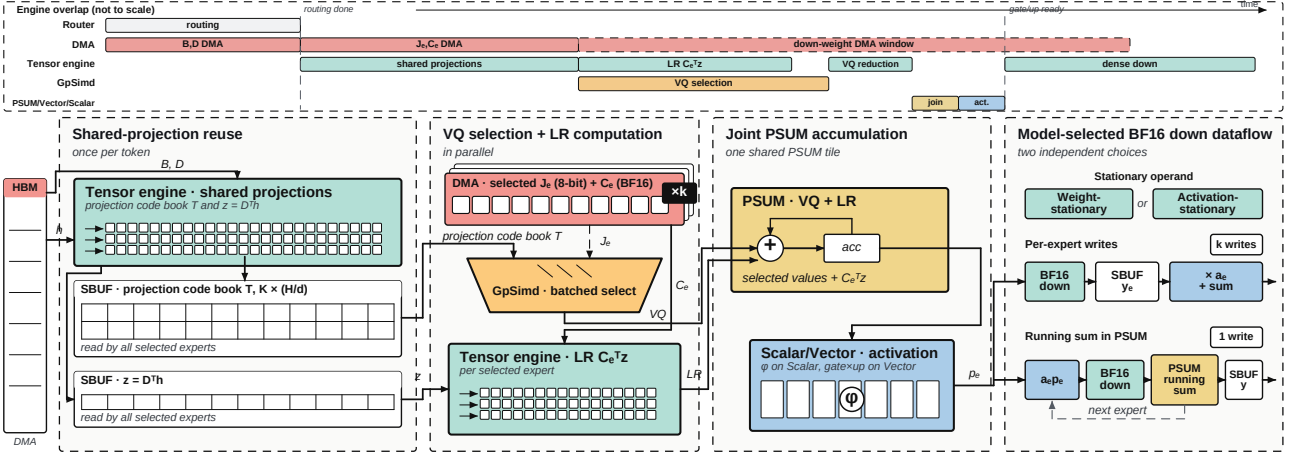


Figure 7. CARDAN’s decoding dataflow. Top: B, D data transfer overlaps routing, expert-private J_e, C_e data transfer overlaps shared projections, down projection weight data transfer overlaps the GpSimd and tensor engine gate/up computation. Bottom: the mechanisms enabling these overlaps.

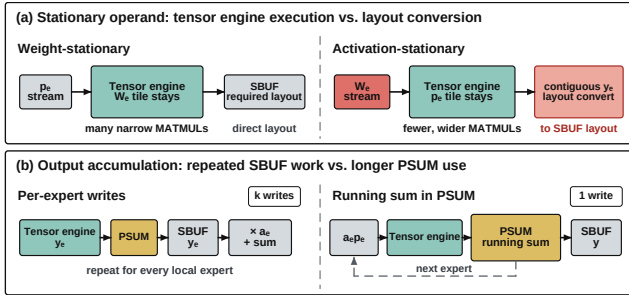


Figure 8. BF16 down projection scheduling choices: (a) weight- versus activation-stationary execution; (b) per-expert SBUF writes versus cross-expert PSUM accumulation.

Specifically, the tensor engine sums the code words selected from T in PSUM, and then accumulates the LR matmul $C_e^T z$ into the same PSUM locations as the code-word summation. Neither LR nor VQ needs to write a standalone output vector to SBUF, reducing memory redundancy in SBUF.

4.3 Schedule of Down Projection

The down projection computes $y_e = W_{\text{down},e}^T p_e$ per activated expert e and $\sum_e a_e y_e$ (named the *accumulated summation*) across the activated experts. After the optimizations in Section 4.2 on the gate- and up-projections, the down-projection, dominated by dense matmul, dominates the remaining time on the tensor engine.

There are two independent dimensions to control the schedule of the down projection. (1) For each activated expert’s down-projection matmul ($y_e = W_{\text{down},e}^T p_e$), which input ($W_{\text{down},e}$ or p_e) should be held stationary in the tensor engine’s systolic array while the other streams through it for better performance? This dimension impacts how many matmuls are issued and whether the data-layout conversion is

needed after matmul. (2) Where is the accumulated summation operated? On SBUF or PSUM? If on SBUF, as soon as y_e for an expert is produced on PSUM, y_e is copied to SBUF and accumulated there, which frees PSUM immediately but costs an extra SBUF write per expert. If on PSUM, the accumulated summation accumulates y_e on PSUM and is then written to SBUF after *all* activated experts are done. Compared with on SBUF, on PSUM, the accumulated summation takes a longer time in PSUM, making PSUM less available to the gate/up projections for better performance.

The two dimensions give four candidate schedules. We decide which schedule performs best using the offline profiling. The offline profiling is feasible. In particular, once an MoE model is selected, the shapes and layout of the down projection, which is needed for deciding the schedule, is determined and do not change across tokens and experts. Hence, the offline profiling results can be used to determine the schedule for online inference. We describe our scheduling decision as follows.

Deciding stationary input. See Figure 8 (a). The execution of weight-stationary generates many skinny matmuls, but directly produces the required SBUF layout. Skinny matmuls come from the multiplication of p_e streaming with the weight tiles resident in the systolic array; the skinny matmul results in the systolic array can be directly used as the down projection without the need of data-layout conversion. The execution of the activation stationary uses fewer, wider matmul, but must convert its output to be aligned with the data-layout of the down projection. CARDAN uses the activation stationary when the tensor engine time the activation stationary saves on the critical path exceeds the layout-conversion time (e.g., the models OLMoE and Nemotron); otherwise, the weight stationary is employed (e.g., the models Qwen3-30B-A3B, DeepSeek-V2-Lite, and Gemma-4-26B-A4B).

Deciding the location of the accumulated summation. See Figure 8(b). Keeping the accumulated summation on PSUM pays off when y_e copying and accumulating on SBUF this method removes is on the critical path (e.g., see the models DeepSeek-V2-Lite, Nemotron, and Gemma 4). When y_e copying and accumulating on SBUF are already hidden behind other operations (e.g., see the models Qwen3-30B-A3B and OLMoE), CARDAN writes y_e to SBUF.

5 Evaluation

CARDAN is implemented as an open-source project with 9.7K lines of code. We evaluate CARDAN with multiple MoE models summarized in Table 2.

5.1 Experimental Setup

All performance experiments run on AWS Trainium3 with four-way tensor parallelism (TP=4), using eight physical NeuronCore-v4s in total. We use logical NeuronCore configuration 2 (LNC=2), which groups two physical NeuronCores into one logical NeuronCore. All models use Neuron SDK 2.31 (neuronx-cc 2.26, NKI 0.5).

We measure batch-one autoregressive decoding unless stated otherwise (Section 5.6), excluding compilation and model loading. We report time per output token (TPOT). For each configuration, we generate 19, 64, and 110 tokens from the same prompt, take the median host time of 24 runs after two warmups at each length, and fit latency as a linear function of generated tokens. TPOT is the slope of this line. Confidence intervals (CIs) are the 2.5th and 97.5th percentiles of TPOT over 10,000 bootstrap resamples of these runs. For device profiles, *device-graph span* denotes the elapsed time from the first to the last operation in a decode-step graph per token.

We compare CARDAN against two baselines.

- **PyTorch/XLA.** This compiler-lowered full-model path preserves the original dense BF16 expert computation and serves as our first baseline without any custom MoE kernel.
- **Dense megakernel.** This optimized MoE implementation based on BF16 is from AWS and works as our primary performance baseline. For Qwen3-30B-A3B and Gemma-4-26B-A4B we use the production megakernel from the public AWS NeuronX Distributed Inference (NxDI) release. Because NxDI fuses MoE decode only for Qwen3-30B-A3B and Gemma-4-26B-A4B, we port the same kernel design to the other three models to obtain a consistent optimized baseline, adapting expert dimensions, activation, and surrounding layers while keeping BF16-based expert computation.

We measure quality by perplexity (PPL) on WikiText-103, evaluated on 32 windows of 512 tokens that are disjoint from the 10,000 256-token windows used for factor initialization

Table 2. MoE models for evaluation.

model	L/L_{MoE}	H/I	E/k	proj.	(d, K, r)
Qwen3-30B-A3B	48/48	2048/768	128/8	gate+up	(8, 256, 384)
DeepSeek-V2-Lite	27/26	2048/1408	64/6	gate+up	(8, 256, 384)
Nemotron-3-Nano-30B	52/23	2688/1856	128/6	up	(8, 256, 512)
OLMoE-1B-7B	16/16	2048/1024	64/8	gate+up	(4, 256, 256)
Gemma-4-26B-A4B	30/30	2816/704	128/8	gate+up	(8, 256, 384)

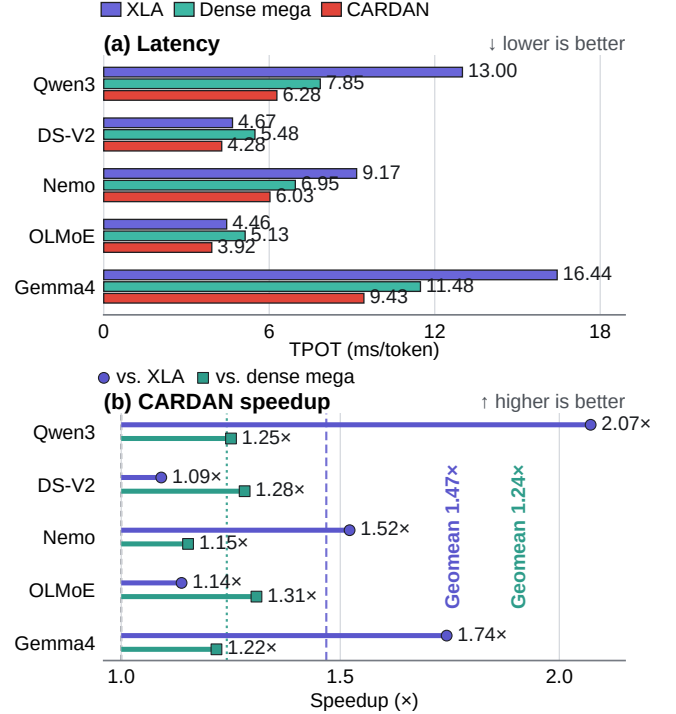


Figure 9. Full-model batch-one decode for deployed configurations: (a) host-observed TPOT; (b) CARDAN speedup (lines: geometric means). Qwen3-30B-A3B and Gemma-4-26B-A4B use the production AWS/NxDI dense kernel; The other baselines port its design.

and distillation. We report each model against its own BF16 teacher measured identically.

5.2 End-to-End Performance

Figure 9 reports end-to-end results. Across the five models, CARDAN is 1.09–2.07× faster than PyTorch/XLA and 1.15–1.31× faster than the dense megakernel. The corresponding geometric-mean speedups are 1.47× and 1.24×. DeepSeek-V2-Lite and Nemotron-3-Nano-30B-A3B additionally fuse dense projections that read the same hidden state into one wider matmul. Section 5.4 separates the gain of this fusion. These gains come from three sources. The hybrid representation reduces the expert-weight data movement that dominates decode. Shared projections are computed once per token instead of once per expert. The overlap schedule hides the remaining transfers and the added indexed selections behind concurrent engine work, keeping their overhead off the critical path.

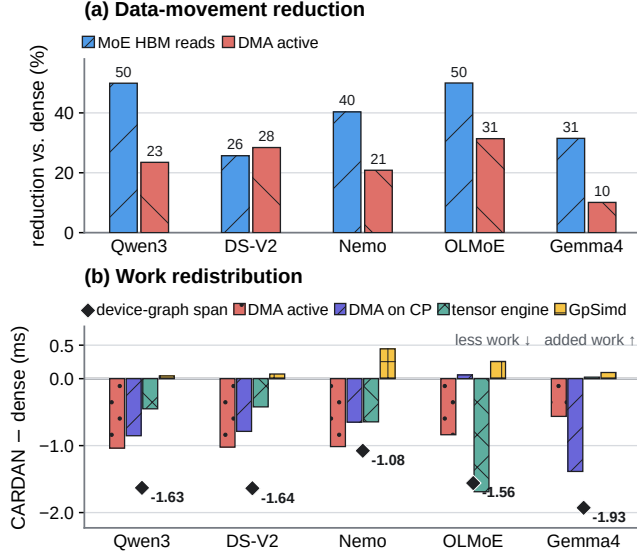


Figure 10. Full-model changes from the dense megakernel: (a) reductions in MoE HBM reads and DMA activity/service; (b) engine-union deltas, with diamonds marking device-graph-span deltas. Intervals overlap.

Table 3. Gemma-4-26B-A4B transition from CARDAN to equal-rate VQ-only. HBM is MB/rank; other values are ms. D/P/G denote DMA service, tensor engine, and non-DMA GpSimd. The final D/G column reports work not hidden by another engine.

representation	HBM	device span	total union			non-overlap D/G
			D	P	G	
CARDAN (8, 512)	1834	8.290	6.182	2.284	0.687	3.287/0.123
VQ only (2, 0)	1763	9.867	6.521	2.559	2.053	3.510/0.852

5.3 Workload Placement and Critical-Path Analysis

We compare CARDAN with the dense megakernel, the baseline which is already a tuned MoE kernel, so the difference isolates the representation and its dataflow. For each model we profile the deployed configuration of Section 5.1 over all layers, running the decode-step graph five times and selecting the trace closest to the median device-graph span. Figure 10 reports one rank of the TP=4 execution. DMA-active time is the total duration during which at least one physical DMA packet is active.

DMA reduction. Figure 10(a) shows that CARDAN reduces MoE-projection HBM reads by 26–50% and DMA-active time by 10–31% across the five models. Total HBM reads fall by 10–32%.

Workload redistribution. Figure 10(b) compares CARDAN with the dense megakernel. CARDAN adds 0.04–0.45 ms of GpSimd work but removes 0.84–1.04 ms of DMA-active time and 0.42–1.69 ms of tensor-engine time (per-engine occupancies, not critical-path time, as the engines overlap). The added GpSimd selections run concurrently with the remaining DMA and tensor-engine work, the device-graph span falls by 1.08–1.93 ms across the five models.

Table 4. Full-model attribution: (A) CARDAN mechanisms on Qwen3-30B-A3B; (B) dense-to-CARDAN latency for DeepSeek and Nemotron with and without surrounding-projection fusion.

A. CARDAN mechanisms on Qwen3-30B-A3B ($d=8, r=384$), 48 layers		
configuration	TPOT (ms)	step reduction
dense megakernel	7.85	—
VQ + LR, per-expert shared proj.	6.90	12.1%
+ shared-projection reuse	6.56	4.9%
+ cross-expert gather batching	6.28	4.3%
B. CARDAN gain with and without fusion		
model	dense → CARDAN (ms/token)	
	without fusion	with fusion
DeepSeek-V2-Lite	5.48 → 4.95	4.82 → 4.28
Nemotron-3	6.94 → 6.25	6.73 → 6.03

Traffic versus runtime overhead. Each VQ block needs one indexed selection on GpSimd, so reducing d increases selection count and can offset the DMA savings. For Gemma-4-26B-A4B, we compare CARDAN at $(d, r) = (8, 512)$ with VQ only at $(d, r) = (2, 0)$, both at 4.00 bits/weight. The $r = 512$ setting is used only for this equal-storage ablation. Setting $r = 0$ removes the LR. To match CARDAN’s storage rate, VQ only reduces d from 8 to 2, which increases the number of indexed selections by 4×. Relative to equal-storage CARDAN, VQ only reads 3.9% fewer HBM bytes but increases GpSimd time by 199%. The GpSimd time exposed on the critical path rises from 0.123 ms in CARDAN to 0.852 ms in VQ only, more than the exposed time of any other compute engine. Device-graph span rises by 19.0%, and TPOT rises from 10.571 to 12.162 ms/token, above the 11.48 ms/token dense baseline. Thus, at a matched storage rate, fewer transferred bytes do not compensate for the added indexed-selection work.

5.4 Kernel Design Ablation

Table 4 separates the gain of CARDAN from model-specific surrounding projection fusion. Part A of the table adds shared-projection reuse and then cross-expert gather batching; Part B reports the dense-to-CARDAN gain for DeepSeek-V2-Lite and Nemotron Nano-30B-A3B with and without their surrounding projection fusion.

The hybrid representation alone reduces Qwen3-30B-A3B TPOT by 12.1%, even when the shared projections are re-computed for every expert. Shared-projection reuse provides a further 4.9% reduction, and cross-expert gather batching provides another 4.3%, for a 20.0% cumulative reduction. A six-layer profile of the same three mechanism variants at $(d, r) = (4, 256)$ shows where each mechanism saves time. Shared-projection reuse reduces tensor-active time from 632 to 562 μ s and DMA-active time from 705 to 643 μ s; cross-expert gather batching cuts GpSimd selection instructions from 385 to 97 and GpSimd time from 616 to 502 μ s.

On Qwen3-30B-A3B, we also tested the down-projection running sum of Section 4.3. It accumulates the activated experts’ down outputs in one PSUM bank and writes the final sum to SBUF once. In isolation, this makes the down step 3.22× faster than the per-expert-copy schedule, but increases

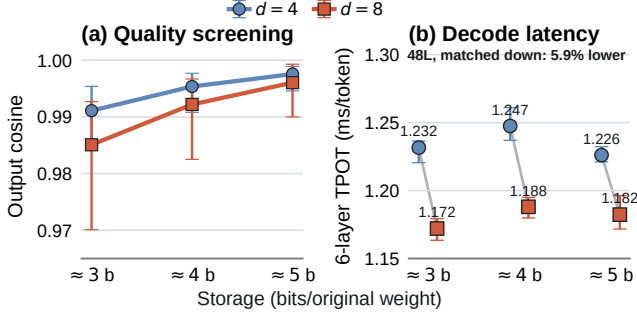


Figure 11. Qwen3-30B-A3B sensitivity: (a) gate/up output cosine at Layers 0/24/47; (b) six-layer TPOT with bootstrap 95% CIs.

48-layer TPOT from 6.278 to 6.432 ms because PSUM remains occupied across the expert loop.

Surrounding-projection fusion is an optimization independent of the CARDAN representation, and it further accelerates CARDAN. Part B separates the two gains by measuring the dense baseline and CARDAN with and without fusion. Fusion reduces latency on both sides, by 11.9% (dense) and 13.6% (CARDAN) on DeepSeek and by 3.1% and 3.6% on Nemotron-3-Nano-30B-A3B. CARDAN stays 1.11–1.13× faster than dense with or without fusion, so its speedup does not depend on fusion.

5.5 Sensitivity and Operating Boundaries

We fix $K = 256$, the largest code book supported by uint8 indices, and store the code book and low-rank factors in BF16. Including both expert-private and amortized shared parameters, the storage rate is

$$b(d, r) = \frac{8}{d} + \frac{16r}{H} + \frac{16r}{EI} + \frac{16Kd}{EHI}$$

bits per original weight (Appendix A.2). Appendix A.3 gives the corresponding per-engine work counts as functions of d , K , and r . We evaluate $d \in \{4, 8\}$ at approximately 3, 4, and 5 bits/weight, choosing r to match their storage within 0.02 bits/weight.

Figure 11 shows the quality-latency tradeoff. $d=8$ lowers TPOT by 3.6–4.8%, whereas $d=4$ retains higher teacher-student output cosine similarity before distillation. At the 4-bit rate, changing (d, r) from $(4, 256)$ to $(8, 384)$ lowers GpSimd time from 495 to 440 μ s despite 4.2% more HBM reads. A separate six-layer, equal-rate VQ-only configuration with $(d, r) = (2, 0)$ removes all low-rank work and reads 12% fewer bytes, yet raises GpSimd time from 440 to 603 μ s and TPOT from about 1.19 to 1.37 ms/token (+14.7%).

On the full 48-layer model, VQ only with $(d, r) = (2, 0)$ reaches 8.075 ms/token, 28.6% slower than CARDAN at $(8, 384)$ (6.278 ms/token) and also slower than the dense BF16 baseline (7.851 ms/token). With $d = 2$, GpSimd selection time exceeds the overlapping DMA time, shifting the decode bottleneck from DMA to GpSimd. Table 3 shows the same shift for Gemma-4-26B-A4B. Thus, although $d = 2$ reduces HBM

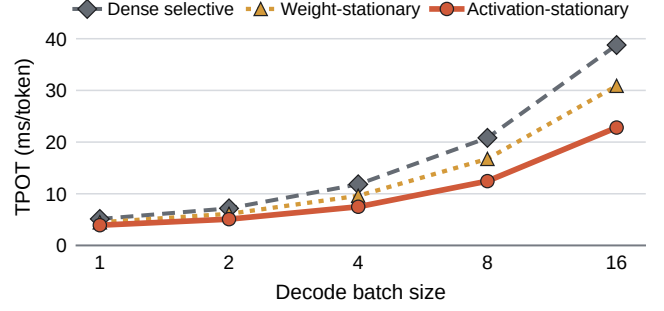


Figure 12. Full-model OLMoE-1B-7B batch scaling. Lower is better.

traffic, its additional indexed-selection work makes it slower overall, so we use $d = 8$.

We also compare against Trainium’s built-in 4-bit-weight, 8-bit-activation (W4A8) quantization, which stores the gate and up weights in the MXFP4 microscaling format while the down projection stays BF16. Under the same full-model TPOT measurement on Qwen3-30B-A3B, W4A8 reaches 11.75 ms/token, slower than the 7.85 of the BF16 dense megakernel. The slowdown comes from a layout boundary rather than from MXFP4 arithmetic. The MX matmul writes a packed layout that only another MX operation can read, so every activated expert’s gate/up output is converted on chip before the BF16 down projection (Appendix A.4).

5.6 Batch Scaling of Selective Decode

We study batch scaling on OLMoE-1B-7B (Table 2), which activates 8 of 64 experts per token. Figure 12 measures the full 16-layer model with $(d, K, r) = (4, 256, 256)$ and the same TPOT method as Figure 9. Each batch receives fixed, distinct prompts. At every batch size, both dense and CARDAN load only the activated experts. CARDAN remains faster at every batch size, growing from 1.31× at batch size 1 (B1) to 1.67× at B8 and 1.70× at B16.

Device profiles reveal the sources of CARDAN’s performance gains. At batch sizes 8 and 16, CARDAN reads 3.33 and 6.13 GB from HBM, respectively, compared with 7.02 and 13.56 GB for the dense baseline. DMA-active time similarly drops from 14.27 to 6.85 ms at batch size 8 and from 27.46 to 13.14 ms at batch size 16. Furthermore, the impact of the down-projection schedule (Section 4.3) becomes more pronounced as the batch size increases. Because both schedules require nearly identical HBM reads, superior operand placement and tensor-engine scheduling enable the activation-stationary schedule to consistently outperform the weight-stationary variant. This speed advantage grows from 12.5% at batch size 1 to 26.2% at 16.

5.7 Model Quality, Compression, and Kernel Fidelity

Table 5 reports perplexity and storage reduction for the five CARDAN models of Section 5.2. The storage figures count

Table 5. Quality and compression of the deployed configurations ($K=256$). “Init.” is closed form; “KD” is knowledge distillation.

model	(d, r)	comp.	teacher	init.	KD	Δ PPL
Qwen3-30B-A3B	(8, 384)	3.94×	9.82	12.51	9.56	−2.7%
DeepSeek-V2-Lite	(8, 384)	3.93×	8.37	12.49	8.22	−1.8%
Nemotron-3-Nano	(8, 512)	3.92×	8.55	9.66	8.29	−3.0%
OLMoE-1B-7B	(4, 256)	3.94×	8.33	9.11	8.24	−1.1%
Gemma-4-26B-A4B	(8, 384)	4.92×	10.98	13.37	10.44	−4.9%

Table 6. Qwen3-30B-A3B quality attribution.. For CARDAN, (8, 384) denotes (d, r). VQ-only $d=8$ removes the LR component; smaller d increases storage. Indexed-selection counts are relative to $d=8$.

representation	training	bits/ weight	indexed selections	PPL	Δ PPL
BF16 teacher	—	16.0	—	9.82	—
shared LR ($r=512$)	closed form	4.08	—	34.91	+255.4%
VQ only ($d=8$)	none	1.00	1×	451.53	+4496.3%
VQ only ($d=4$)	none	2.00	2×	22.47	+128.7%
VQ only ($d=2$)	none	4.00	4×	10.44	+6.3%
CARDAN (8, 384)	closed form	4.06	1×	12.51	+27.3%
CARDAN (8, 384)	distillation	4.06	1×	9.56	−2.7%

the expert-private indices and coefficient matrices, the shared basis and code book, and padding and alignment. All models use $K=256$ with uint8 indices and the (d, r) of Table 2, which shrinks the gate and up weights by 3.92–4.92× on five models. Closed-form initialization recovers most of the quality lost to the coarse VQ. Short end-to-end distillation then restores PPL to the BF16-teacher level across all five models. On four zero-shot downstream tasks, CARDAN limits Qwen3-30B-A3B’s accuracy loss to at most 3.2 percentage points relative to original BF16 model (Appendix A.5).

Kernel fidelity. We separately verify the numerical fidelity of the Trainium implementation against an FP32 PyTorch reference. For each model, we execute one MoE layer on Trainium using the fitted $\mathcal{B}$, J_e , D , and C_e and the model’s actual dimensions, including the gate, up, and down projections and the weighted sum over the activated experts. Compared with the same computation in FP32 PyTorch, the normalized maximum error, $\max |\hat{y} - y| / \max |y|$, is 3.40×10^{-3} for Qwen3-30B-A3B, 4.70×10^{-3} for DeepSeek-V2-Lite, 2.36×10^{-3} for Nemotron-3-Nano-30B-A3B, 4.90×10^{-3} for OLMoE-1B-7B, and 2.79×10^{-3} for Gemma-4-26B-A4B. The Nemotron-3-Nano-30B-A3B test covers its up-only ReLU² experts, whose hidden dimension is padded for the VQ blocks.

Table 6 compares stronger VQ-only configurations with $d = 4$ and $d = 2$ against the deployed $d = 8$ ablation with the LR removed. VQ-only with $d = 2$ reaches PPL 10.44, 6.3% above the BF16 teacher.

6 Related Work

Scratchpad accelerators and workload placement. From systolic arrays and flexible-dataflow designs [6, 22, 24, 25, 35] to modern commercial systems (TPU, Trainium, Ascend, Tenstorrent, Gaudi) [3, 15, 18, 20, 41], accelerators couple dense compute with software-managed on-chip memory and heterogeneous engines, but generally take the stored tensor

representation as given. Our question is complementary: how an MoE expert format changes the bytes and operations assigned to concurrent engines. LLM decode is memory-bound under the roofline due to low weight reuse [32, 46], and overlap determines which work reaches latency [31]. Other systems change the memory path: SN40L streams composition-of-experts models through tiered memory [33], NPU-PIM partitions bandwidth-heavy operators [16, 49], and near-data MoE moves activations to cold experts [23]. With the commercial accelerator fixed, CARDAN instead co-designs the expert representation and decode dataflow around DMA, tensor-engine work, and indexed-selection work.

Low-rank MoE factorization. Activation-aware SVD [45], shared-basis MoE factorization [5, 28], and architectural shared experts [8] reuse dense factors across experts. Shared factors are regular dense contractions that cut expert-private traffic, but confining every expert to one subspace caps capacity—the quality wall evaluated in Section 5.7. CARDAN retains this dataflow only to correct the VQ component’s output error, not to approximate the whole weight.

Weight vector quantization. Scalar post-training quantization [12], product quantization [21], and multi-code book, lattice, or trellis LLM quantizers [10, 27, 42–44] preserve high-rank structure while optimizing accuracy at a storage rate for GPU decode. RQ-MoE builds input-dependent code books for compressed embeddings [52]; CARDAN compresses static expert weights with expert-private index maps. Low-rank quantization-error corrections fit factors per matrix [38, 48, 50, 51], with LQER motivating dense correction over irregular high-precision gathers. We pool post-VQ residuals into one shared basis and, unlike bits-only comparisons, price indexed selection against DMA overlap: finer code vectors save HBM traffic but can make selection the bottleneck.

MoE serving and distillation. QMoE co-designs a sub-bit dictionary with GPU decode kernels [11]; DeepSpeed-MoE and MegaBlocks optimize placement, communication, and routed GEMMs [13, 37]; and pre-gated MoE hides expert offload through prefetching [19]. CARDAN instead decides which representation state is shared, routed, and decoded by each engine. Knowledge distillation [17] recovers the final quality gap but is not our contribution.

7 Conclusion

On an STA, an MoE expert representation defines its hardware dataflow. CARDAN co-designs the representation with decoder execution to break the strict routing–transfer–compute dependency chain: layer-shared factors expose routing independent work, while expert-private indices and coefficients preserve expert-specific capacity on the routing dependent path. This separation allows shared-factor transfers to overlap routing, shared projections to overlap expert-private

transfers, and regular and indexed work to execute concurrently on the tensor engine and GpSimd. Across five MoE families, this co-designed dataflow delivers $1.15\text{--}1.31\times$ decode speedup with PPL at or below that of the BF16 teachers. More broadly, CARDAN shows that model representation and STA dataflow should be designed together around the critical path, data movement, scratchpad residency, heterogeneous engine work, and synchronization.

References

- [1] Amazon Web Services. 2026. Introduction to Direct Memory Access (DMA) with NKL. AWS Neuron Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/v2.31.0/nki/get-started/about/nki-dma-overview.html> Accessed August 23, 2026.
- [2] Amazon Web Services. 2026. NKL Direct Allocation Developer Guide. AWS Neuron Documentation. https://awsdocs-neuron.readthedocs-hosted.com/en/v2.26.1/nki/nki_direct_allocation_guide.html Accessed August 22, 2026.
- [3] Amazon Web Services. 2026. Trainium3 Architecture Guide for NKL. AWS Neuron Documentation. https://awsdocs-neuron.readthedocs-hosted.com/en/v2.30.0/nki/guides/architecture/trainium3_arch.html Accessed August 15, 2026.
- [4] Thomas Benz, Michael Rogenmoser, Paul Scheffler, Samuel Riedel, Alessandro Ottaviano, Andreas Kurth, Torsten Hoefler, and Luca Benini. 2024. A High-Performance, Energy-Efficient Modular DMA Engine Architecture. *IEEE Trans. Comput.* 73, 1 (2024), 263–277. <https://doi.org/10.1109/TC.2023.3329930>
- [5] Xiaodong Chen, Mingming Ha, Zhenzhong Lan, Jing Zhang, and Jianguo Li. 2025. MoBE: Mixture-of-Basis-Experts for Compressing MoE-based LLMs. arXiv:2508.05257. <https://doi.org/10.48550/arXiv.2508.05257>
- [6] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 367–379. <https://doi.org/10.1109/ISCA.2016.40>
- [7] Yu-Hsin Chen, Tien-Ju Yang, Joel Emer, and Vivienne Sze. 2019. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 2 (2019), 292–308. <https://doi.org/10.1109/JETCAS.2019.2910232>
- [8] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jishi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 1280–1297. <https://doi.org/10.18653/v1/2024.acl-long.70>
- [9] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Helen Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 6. Curran Associates, Inc., Red Hook, NY, USA, 224–238. https://proceedings.mlsys.org/paper_files/paper/2024/hash/698cfaf72a208aef2e78bcac55b74328-Abstract-Conference.html
- [10] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Vienna, Austria, 12284–12303. <https://proceedings.mlr.press/v235/egiazarian24a.html>
- [11] Elias Frantar and Dan Alistarh. 2024. QMoE: Sub-1-Bit Compression of Trillion Parameter Models. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 6. Curran Associates, Inc., Red Hook, NY, USA, 439–451. https://proceedings.mlsys.org/paper_files/paper/2024/hash/c74b624843218d9b6713fcf299d6d5e4-Abstract-Conference.html
- [12] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=tcbBPnfwxS>
- [13] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 5. Curran Associates, Inc., Red Hook, NY, USA, 288–304. https://proceedings.mlsys.org/paper_files/paper/2023/hash/5a54f79333768effe7e8927bccffe40-Abstract-mlsys2023.html
- [14] Google DeepMind. 2026. Gemma-4-26B-A4B. Hugging Face model repository. <https://huggingface.co/google/gemma-4-26B-A4B> Accessed August 31, 2026.
- [15] Habana Labs. 2022. Gaudi2 White Paper. Intel. <https://www.intel.com/content/www/us/en/content-details/784827/gaudi-2-white-paper.html>
- [16] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 722–737. <https://doi.org/10.1145/3620666.3651380>
- [17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. arXiv:1503.02531. <https://doi.org/10.48550/arXiv.1503.02531>
- [18] Huawei Technologies Co., Ltd. 2026. Hardware Architecture Abstraction. Ascend C Operator Development Guide, CANN Commercial Edition 8.0.0. https://www.hiascend.com/document/detail/en/canncommercial/800/opdevg/Ascendcopdevg/atlas_ascendc_10_0015.html Accessed August 15, 2026.
- [19] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 1018–1031. <https://doi.org/10.1109/ISCA59077.2024.00078>
- [20] JAX Authors. 2026. TPU Pipelining. JAX Documentation. <https://docs.jax.dev/en/latest/pallas/tpu/pipelining.html> Accessed August 15, 2026.
- [21] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [22] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. 2017. In-Datcenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3140659.3080246>
- [23] Taehyun Kim, Kwanseok Choi, Youngmook Cho, Jaehoon Cho, Hyuk-Jae Lee, and Jaewoong Sim. 2024. MoNDE: Mixture of Near-Data Experts for Large-Scale Sparse Models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3649329.3655951>
- [24] Hsiang-Tsung Kung. 1982. Why Systolic Architectures? *Computer* 15, 1 (1982), 37–46. <https://doi.org/10.1109/MC.1982.1653825>
- [25] Hyoungjun Kwon, Ananda Samajdar, and Tushar Krishna. 2018. MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 461–475. <https://doi.org/10.1145/3173162.3173176>
- [26] Yoseph Linde, Andres Buzo, and Robert M. Gray. 1980. An Algorithm for Vector Quantizer Design. *IEEE Transactions on Communications* 28, 1 (1980), 84–95. <https://doi.org/10.1109/TCOM.1980.1094577>

- [27] Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang. 2024. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Miami, FL, USA, 8181–8196. <https://doi.org/10.18653/v1/2024.emnlp-main.467>
- [28] Zehua Liu, Han Wu, Ruifeng She, Xiaojin Fu, Xiongwei Han, Tao Zhong, and Mingxuan Yuan. 2025. MoLAE: Mixture of Latent Experts for Parameter-Efficient Language Models. arXiv:2503.23100. <https://doi.org/10.48550/arXiv.2503.23100>
- [29] NVIDIA. 2025. NVIDIA-Nemotron-3-Nano-30B-A3B-BF16. Hugging Face model repository. <https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16> Accessed August 31, 2026.
- [30] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ñiigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. <https://doi.org/10.1109/ISCA59077.2024.00019>
- [31] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking and Triggering for Fine-grained Overlap of Compute and Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 1146–1164. <https://doi.org/10.1145/3620665.3640410>
- [32] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently Scaling Transformer Inference. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 5. Curran Associates, Inc., Red Hook, NY, USA, 606–624. https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dfca2780-Abstract-mlsys2023.html
- [33] Raghu Prabhakar, Ram Sivaramakrishnan, Darshan Gandhi, Yun Du, Mingran Wang, Xiangyu Song, Kejie Zhang, Tianren Gao, Angela Wang, Xiaoyan Li, Yongning Sheng, Joshua Brot, Denis Sokolov, Apurv Vivek, Calvin Leung, Arjun Sabnis, Jiayu Bai, Tuowen Zhao, Mark Gottscho, David Jackson, Mark Luttrell, Manish K. Shah, Zhengyu Chen, Kaizhao Liang, Swayambhoo Jain, Urmish Thakker, Dawei Huang, Sumti Jairath, Kevin J. Brown, and Kunle Olukotun. 2024. SambaNova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 1353–1366. <https://doi.org/10.1109/MICRO61859.2024.00100>
- [34] Eric Qin, Geonhwa Jeong, William Won, Sheng-Chun Kao, Hyoukjun Kwon, Sudarshan Srinivasan, Dipankar Das, Gordon Euhyun Moon, Sivasankaran Rajamanickam, and Tushar Krishna. 2021. Extending Sparse Tensor Accelerators to Support Multiple Compression Formats. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 1014–1024. <https://doi.org/10.1109/IPDPS49936.2021.00110>
- [35] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 58–70. <https://doi.org/10.1109/HPCA47549.2020.00015>
- [36] Qwen Team. 2025. Qwen3-30B-A3B. Hugging Face model repository. <https://huggingface.co/Qwen/Qwen3-30B-A3B> Accessed August 31, 2026.
- [37] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, Baltimore, MD, USA, 18332–18346. <https://proceedings.mlr.press/v162/rajbhandari22a.html>
- [38] Rajarshi Saha, Naomi Sagan, Varun Srivastava, Andrea J. Goldsmith, and Mert Pilanci. 2024. Compressing Large Language Models using Low Rank and Low Precision Decomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 88981–89018. <https://doi.org/10.52202/079017-2823>
- [39] Selma Saidi, Pranav Tendulkar, Thierry Lepley, and Oded Maler. 2012. Optimizing Explicit Data Transfers for Data Parallel Applications on the Cell Architecture. *ACM Transactions on Architecture and Code Optimization* 8, 4 (2012), 37:1–37:20. <https://doi.org/10.1145/2086696.2086716>
- [40] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-based LLM Serving using Proactive Caching. arXiv:2410.22134. <https://doi.org/10.48550/arXiv.2410.22134>
- [41] Tenstorrent. 2026. TT Architecture and Metalium Guide. TT-Metalium Documentation. https://github.com/tenstorrent/tt-metal/blob/main/METALIUM_GUIDE.md Accessed August 15, 2026.
- [42] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Vienna, Austria, 48630–48656. <https://proceedings.mlr.press/v235/tseng24a.html>
- [43] Albert Tseng, Qingyao Sun, David Hou, and Christopher De Sa. 2024. QTIP: Quantization with Trellises and Incoherence Processing. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 59597–59620. <https://doi.org/10.52202/079017-1904>
- [44] Mart van Baalen, Andrey Kuzmin, Markus Nagel, Peter Couperus, Cedric Bastoul, Eric Mahurin, Tijmen Blankevoort, and Paul Whatmough. 2024. GPTVQ: The Blessing of Dimensionality for LLM Quantization. arXiv:2402.15319. <https://doi.org/10.48550/arXiv.2402.15319>
- [45] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. 2025. SVD-LLM: Truncation-Aware Singular Value Decomposition for Large Language Model Compression. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=LNYIUouhdt>
- [46] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. <https://doi.org/10.1145/1498765.1498785>
- [47] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Offloading-Efficient MoE Model Serving. arXiv:2401.14361. <https://doi.org/10.48550/arXiv.2401.14361>
- [48] Zhewei Yao, Xiaoxia Wu, Cheng Li, Stephen Youn, and Yuxiong He. 2023. ZeroQuant-V2: Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation. arXiv:2303.08302. <https://doi.org/10.48550/arXiv.2303.08302>
- [49] Sungmin Yun, Kwanhee Kyung, Juhwan Cho, Jaewan Choi, Jongmin Kim, Byeongho Kim, Sukhan Lee, Kyomin Sohn, and Jung Ho Ahn. 2024. Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 1429–1443. <https://doi.org/10.1109/MICRO61859.2024.00105>
- [50] Cheng Zhang, Jianyi Cheng, George A. Constantinides, and Yiren Zhao. 2024. LQER: Low-Rank Quantization Error Reconstruction for LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, Vienna, Austria, 58763–58779. <https://proceedings.mlr.press/v235/zhang24j.html>

- [51] Cheng Zhang, Jeffrey T. H. Wong, Can Xiao, George A. Constantinides, and Yiren Zhao. 2025. QERA: An Analytical Framework for Quantization Error Reconstruction. International Conference on Learning Representations (ICLR). <https://openreview.net/forum?id=LB5cKhgOTu>
- [52] Zhengjia Zhong, Shuyan Ke, Zaizhou Lin, Jiaqi Song, Hongyi Lan, and Hui Li. 2026. RQ-MoE: Residual Quantization via Mixture of Experts for Efficient Input-Dependent Vector Compression. arXiv:2605.14359. <https://doi.org/10.48550/arXiv.2605.14359>

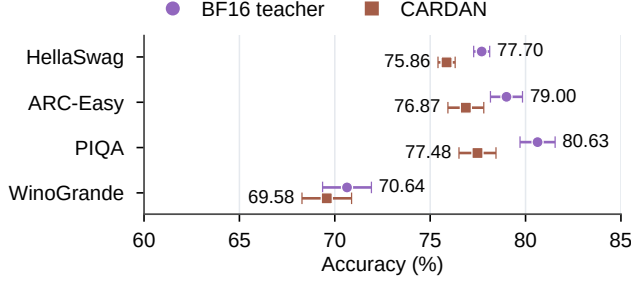


Figure 13. Zero-shot accuracy of Qwen3-30B-A3B. HellaSwag, ARC-Easy, and PIQA use length-normalized accuracy; WinoGrande uses unnormalized accuracy.

A Appendix

A.1 Whitened Shared-Basis Correction

Single-matrix low-rank quantization-error correction is established [38, 48], including an activation-weighted closed form in QERA [51]. Equation (5) instead fits one subspace to the pooled expert Gram, yielding a layer-shared D .

For completeness, define the post-VQ residual $\Delta_e = W_e - Q_{\mathcal{B}}(J_e)$. The shared-basis correction ($D, \{C_e\}$) minimizes the activation-weighted output error

$$\min_{D, \{C_e\}} \sum_e w_e \left\| \Sigma_x^{1/2} (\Delta_e - DC_e) \right\|_F^2, \quad \Sigma_x = \mathbb{E}[hh^\top]. \quad (4)$$

Let $R = \text{chol}(\Sigma_x)$, with $\Sigma_x = RR^\top$. The route-aware variant of Eq. (4) uses $w_e = \mathbb{E}[1(e \in \mathcal{S}(h))a_e^2]$; the default sets all $w_e = 1$. Let

$$U_r = \text{top-}r \left(R^\top \left(\sum_e w_e \Delta_e \Delta_e^\top \right) R \right). \quad (5)$$

The shared basis and expert coefficient matrices are

$$D = R^{-T} U_r, \quad C_e = (D^\top \Sigma_x D)^{-1} D^\top \Sigma_x \Delta_e. \quad (6)$$

To see optimality, whiten each post-VQ residual as $A_e = R^\top \Delta_e$. For fixed VQ factors, Eq. (4) becomes a weighted shared-subspace approximation of the matrices A_e . The Ky Fan/Eckart–Young result selects the top- r eigenspace of their pooled weighted Gram, giving Eq. (5); ordinary least squares then gives Eq. (6). This proves optimality for the calibration objective under a common activation covariance. It does not imply monotonic validation PPL.

A.2 Exact rate accounting

The total storage rate per original weight, including the expert-private J_e and C_e and the layer-shared $\mathcal{B}$ and D , is

$$b_{\text{total}} = \frac{\log_2 K}{d} + \frac{b_{Cr}}{H} + \frac{b_{\mathcal{B}} K d + b_D H r}{E H I}. \quad (7)$$

The final term is amortized across E experts but is included in every reported model-size number. The implementation uses one-byte indices for $K = 256$ and BF16 for deployed $\mathcal{B}$,

C_e , and D ; training checkpoints retain FP32 master copies that are not counted as inference storage.

A.3 Exact Engine-Work Counts

Table 7 summarizes how the representation parameters affect the leading-order decode work on each engine for one projection. Shared counts apply once per token, while expert-private counts apply to each selected expert. Let $b_J(K)$ denote the deployed index width; $b_J(256) = 8$ in our implementation, and the width changes only when K crosses an encoding boundary. For compactness, Table 7 uses PE to denote the tensor engine.

TABLE 7

REPRESENTATION PARAMETERS AND LEADING-ORDER DECODE WORK FOR ONE PROJECTION.

scope / engine	operation	count	trend
shared PE	code book projection	HK MACs	$K \uparrow \Rightarrow \uparrow$
shared PE	basis projection $D^\top h$	Hr MACs	$r \uparrow \Rightarrow \uparrow$
shared DMA	load $\mathcal{B}$	$16Kd$ bits	$d \uparrow \Rightarrow \uparrow, K \uparrow \Rightarrow \uparrow$
shared DMA	load D	$16Hr$ bits	$r \uparrow \Rightarrow \uparrow$
expert-private DMA	load J_e	$\frac{HI}{d} b_J(K)$ bits	$d \uparrow \Rightarrow \downarrow, b_J \uparrow \Rightarrow \uparrow$
expert-private DMA	load C_e	$16Ir$ bits	$r \uparrow \Rightarrow \uparrow$
expert-private GpSimd	indexed selection	$\frac{HI}{d}$ selections	$d \uparrow \Rightarrow \downarrow$
expert-private PE	VQ reduction	$\frac{HI}{d}$ adds	$d \uparrow \Rightarrow \downarrow$
expert-private PE	low-rank $C_e^\top z$	Ir MACs	$r \uparrow \Rightarrow \uparrow$

A.4 Native MX at the BF16 Down Boundary

Our native selective Qwen W4A8 control preserves top-8 routing at TP=4 and LNC=2: gate/up weights use MXFP4, their shared BF16 input is quantized online to MXFP8 once per layer, and down remains BF16. The W4A8 control therefore targets the same routed gate/up traffic as CARDAN without evaluating all experts or changing the scheduler.

The MX primitive emits an accelerator-specific x4-packed representation (the 4P+Q layout in our implementation), which can remain in SBUF for an MX consumer. BF16 down instead requires an on-chip conversion of every selected expert’s output; this is not an HBM materialization. In the compiled six-layer control, replacing the MX down consumer with BF16 increases STREAMTRANPOSE operations from 1 to 33 and TENSORCOPY from 34 to 110 per logical kernel. We report this as a layout-boundary result, not as evidence that MXFP4 arithmetic is intrinsically slow.

A.5 Downstream Task Accuracy

We evaluate Qwen3-30B-A3B zero-shot on HellaSwag, ARC-Easy, PIQA, and WinoGrande-XL using lm-eval-harness v0.4.13. The BF16 teacher and CARDAN checkpoint (d, r) = (8, 384) share the same tokenizer, request ordering. Figure 13 shows the comparison.